机器视觉 作业6

一、 题目要求

1. 利用第5次作业计算出的右边图像中红色ROI框的方法，编程实现连续多张输入图像中红色ROI方框中心坐标(rcx,rcy)和旋转角度θcr，绿色方框可以用Halcon模板匹配函数find_ncc_model ()实现。
   要求：提交如上图所示的显示图像及程序源代码。
2. 实现两层的NCC模板匹配算法
   输出目标的位置坐标(x,y)，NCC值以及计算时间。
   编程语言C/C++。

二、 算法设计与解题思路

本作业采用 C++ 与 OpenCV 从底层实现。为了同时满足题目 1 和题目 2 的要求，本程序摒弃了 Halcon 现成的算子，自主实现了一套带有旋转搜索功能和两层金字塔加速的 NCC（归一化互相关）模板匹配算法。

1. 两层金字塔 NCC 匹配 (解答题2)：
   - 顶层 (Level 1) 粗搜： 将原图与模板尺寸缩小为原来的 $1/2$，在 -10° 到 $+10^\circ$ 范围内以 1.0° 为步长进行全图遍历匹配，快速锁定极值点与大致角度，大幅缩减计算量。
   - 底层 (Level 0) 精搜： 将粗搜坐标映射回原图尺度，在目标点周围 15×15 像素的局部 ROI 内，以 0.1° 为步长进行精确的角度与坐标拟合，最终输出带旋转角度的绿框坐标、NCC 得分及毫秒级耗时。
2. ROI 坐标推导与绘制 (解答题1)：
   - 利用作业 5 提取的基准绿框中心与红框中心的相对偏移向量。
   - 根据精搜得到的当前模板角度差值 $\Delta\theta$，利用二维旋转矩阵对相对偏移向量进行旋转变换。
   - 将旋转后的向量叠加至当前绿框中心，推导出红框 (ROI) 的精确中心坐标 (r_{cx}, r_{cy}) 及旋转角度 θ_{cr}，并利用 `RotatedRect` 将双框绘制并保存。

三、 程序源代码 (C++)

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <chrono>
#include <opencv2/opencv.hpp>
#include <cmath>

#ifdef _WIN32
#include <windows.h>
#endif

using namespace std;
using namespace cv;

// ================ 从作业5中得出的已知常量 ================
```

```cpp
const double T_RX = 440.495;
const double T_RY = 114.496;

const double R_RX = 436.762;
const double R_RY = 130.551;
const double THETA_RR = 0.471;

// 基准偏移量 (红框相对于绿框中心的向量)
const double DELTA_X_REF = R_RX - T_RX;
const double DELTA_Y_REF = R_RY - T_RY;

// ================= 辅助函数：旋转图像防裁剪 =================
Mat rotateImage(const Mat& src, double angle, Point2f& new_center) {
    // OpenCV 的 getRotationMatrix2D：angle为正是逆时针(CCW)
    Point2f center(src.cols / 2.0f, src.rows / 2.0f);
    Mat rot_mat = getRotationMatrix2D(center, angle, 1.0);

    Rect2f bbox = RotatedRect(Point2f(), src.size(), angle).boundingRect2f();
    rot_mat.at<double>(0, 2) += bbox.width / 2.0 - src.cols / 2.0;
    rot_mat.at<double>(1, 2) += bbox.height / 2.0 - src.rows / 2.0;

    Mat dst;
    warpAffine(src, dst, rot_mat, bbox.size(), INTER_LINEAR, BORDER_CONSTANT,
Scalar(0));
    new_center = Point2f(bbox.width / 2.0f, bbox.height / 2.0f);
    return dst;
}

int main() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif

    cout << "========== 作业6：两层金字塔 NCC 与 ROI 追踪 (终极修正版) ==========\n" <<
endl;

    Mat tpl = imread("model4.bmp", IMREAD_GRAYSCALE);
    if (tpl.empty()) {
        cout << "[错误] 找不到模板图像 model4.bmp! " << endl;
        return -1;
    }

    vector<string> image_names = {
        "1-3.jpg", "1-4.jpg", "1-15.jpg", "1-17.jpg",
        "10.jpg", "11.jpg", "12.jpg", "23.jpg", "24.jpg",
        "25.jpg", "36.jpg", "37.jpg", "38.jpg", "47.jpg",
        "48.jpg", "49.jpg"
    };
```

```cpp
    for (const auto& img_name : image_names) {
        Mat img_color = imread(img_name, IMREAD_COLOR);
        if (img_color.empty()) continue;

        Mat img;
        cvtColor(img_color, img, COLOR_BGR2GRAY);
        cout << ">>> 正在处理: " << img_name << " <<<" << endl;

        auto start_time = chrono::high_resolution_clock::now();

        // =============== 顶层 Level 1: 粗搜 ===============
        Mat img_lvl1, tpl_lvl1;
        resize(img, img_lvl1, Size(), 0.5, 0.5);
        resize(tpl, tpl_lvl1, Size(), 0.5, 0.5);

        double best_score_lvl1 = -1.0;
        Point best_loc_lvl1;
        double best_angle_lvl1 = 0;

        for (double a = -15.0; a <= 15.0; a += 1.0) {
            Point2f dummy_center;
            Mat rot_tpl = rotateImage(tpl_lvl1, a, dummy_center);
            if (rot_tpl.cols > img_lvl1.cols || rot_tpl.rows > img_lvl1.rows)
continue;

            Mat res;
            matchTemplate(img_lvl1, rot_tpl, res, TM_CCOEFF_NORMED);
            double minV, maxV; Point minL, maxL;
            minMaxLoc(res, &minV, &maxV, &minL, &maxL);

            if (maxV > best_score_lvl1) {
                best_score_lvl1 = maxV;
                best_loc_lvl1 = maxL;
                best_angle_lvl1 = a;
            }
        }

        // =============== 底层 Level 0: 精搜 ===============
        int coarse_x = best_loc_lvl1.x * 2;
        int coarse_y = best_loc_lvl1.y * 2;
        int search_range = 25;

        int roi_x = max(0, coarse_x - search_range);
        int roi_y = max(0, coarse_y - search_range);
        int roi_w = min(img.cols - roi_x, tpl.cols * 2 + search_range * 2);
        int roi_h = min(img.rows - roi_y, tpl.rows * 2 + search_range * 2);
        Mat img_roi = img(Rect(roi_x, roi_y, roi_w, roi_h));

        double final_score = -1.0;
```

```cpp
        Point final_loc;
        double final_angle = best_angle_lvl1; // CCW 为正
        Point2f final_center_offset;

        for (double a = best_angle_lvl1 - 2.0; a <= best_angle_lvl1 + 2.0; a += 0.1) {
            Point2f center_offset;
            Mat rot_tpl = rotateImage(tpl, a, center_offset);
            if (rot_tpl.cols > img_roi.cols || rot_tpl.rows > img_roi.rows) continue;

            Mat res;
            matchTemplate(img_roi, rot_tpl, res, TM_CCOEFF_NORMED);
            double minV, maxV; Point minL, maxL;
            minMaxLoc(res, &minV, &maxV, &minL, &maxL);

            if (maxV > final_score) {
                final_score = maxV;
                final_loc = maxL;
                final_angle = a;
                final_center_offset = center_offset;
            }
        }

        auto end_time = chrono::high_resolution_clock::now();
        chrono::duration<double, std::milli> elapsed = end_time - start_time;

        // ============== 数学逻辑核心修正区 ==============

        // 1. 获取绿框（当前模板）的实际中心点 t_cx, t_cy
        double matched_x = final_loc.x + roi_x;
        double matched_y = final_loc.y + roi_y;
        double t_cx = matched_x + final_center_offset.x;
        double t_cy = matched_y + final_center_offset.y;

        // 2. 推导红框（ROI）坐标
        // final_angle 也是相对于基准的旋转差值（以逆时针 CCW 为正）
        double rad = final_angle * CV_PI / 180.0;

        // 图像是 Y 轴向下的坐标系！
        // 在 Y 轴向下的坐标系中，向量 (x, y) 逆时针旋转 rad 弧度的纯数学公式为：
        // x' = x*cos(rad) + y*sin(rad)
        // y' = -x*sin(rad) + y*cos(rad)
        double rx_rot = DELTA_X_REF * cos(rad) + DELTA_Y_REF * sin(rad);
        double ry_rot = -DELTA_X_REF * sin(rad) + DELTA_Y_REF * cos(rad);

        double r_cx = t_cx + rx_rot;
        double r_cy = t_cy + ry_rot;

        // 3. 计算红框新角度
        double r_angle = THETA_RR + final_angle;
```

```
        // ================= 终端输出解答 =================
        cout << "  [NCC 搜索结果 (第2题)]" << endl;
        cout << "   - 绿框中心 (tcx, tcy): (" << t_cx << ", " << t_cy << ")" << endl;
        cout << "   - 匹配得分 (NCC值)   : " << final_score << endl;
        cout << "   - 算法耗时           : " << elapsed.count() << " ms" << endl;

        cout << "  [ROI 推导结果 (第1题)]" << endl;
        cout << "   - 红框中心 (rcx, rcy): (" << r_cx << ", " << r_cy << ")" << endl;
        cout << "   - 红框角度 (theta_cr): " << r_angle << " 度\n" << endl;

        // ================= 绘制双框并保存 =================
        // OpenCV 的 RotatedRect 函数是以 顺时针(CW) 为正角度!
        // 因为我们算出的 final_angle 是逆时针(CCW)为正，所以必须传入 -final_angle 才能正
确绘制
        RotatedRect green_rect(Point2f(t_cx, t_cy), tpl.size(), -final_angle);
        Point2f v_g[4]; green_rect.points(v_g);
        for (int i = 0; i < 4; i++) line(img_color, v_g[i], v_g[(i + 1) % 4],
Scalar(0, 255, 0), 2);

        // 红框尺寸缩小到贴合十字横条 (120x25)，并传相反的角度
        RotatedRect red_rect(Point2f(r_cx, r_cy), Size2f(120.0f, 25.0f), -r_angle);
        Point2f v_r[4]; red_rect.points(v_r);
        for (int i = 0; i < 4; i++) line(img_color, v_r[i], v_r[(i + 1) % 4],
Scalar(0, 0, 255), 2);

        string save_name = "Result_" + img_name;
        imwrite(save_name, img_color);
    }

    cout << "================= 所有图像处理及保存完毕 =================" << endl;
    return 0;
}
```

四、 运行结果数据

```
========== 作业6：两层金字塔 NCC 与 ROI 追踪 ==========

>>> 正在处理: 1-3.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (315.164, 216.187)
   - 匹配得分 (NCC值)   : 0.91467
   - 算法耗时           : 133.945 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (311.459, 232.248)
   - 红框角度 (theta_cr): 0.571 度

>>> 正在处理: 1-4.jpg <<<
```

```
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (313.164, 215.187)
   - 匹配得分 (NCC值)   : 0.91147
   - 算法耗时           : 144.805 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (309.459, 231.248)
   - 红框角度 (theta_cr): 0.571 度

>>> 正在处理: 1-15.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (294.371, 324.652)
   - 匹配得分 (NCC值)   : 0.718158
   - 算法耗时           : 111.855 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (292.166, 340.987)
   - 红框角度 (theta_cr): 5.871 度

>>> 正在处理: 1-17.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (314.938, 182.669)
   - 匹配得分 (NCC值)   : 0.708238
   - 算法耗时           : 148.941 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (313.246, 199.066)
   - 红框角度 (theta_cr): 7.671 度

>>> 正在处理: 10.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (503, 362)
   - 匹配得分 (NCC值)   : 0.982103
   - 算法耗时           : 75.6244 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (499.267, 378.055)
   - 红框角度 (theta_cr): 0.471 度

>>> 正在处理: 11.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (502, 319)
   - 匹配得分 (NCC值)   : 0.98749
   - 算法耗时           : 73.2433 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (498.267, 335.055)
   - 红框角度 (theta_cr): 0.471 度

>>> 正在处理: 12.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (459, 320)
   - 匹配得分 (NCC值)   : 0.986086
   - 算法耗时           : 80.5917 ms
```

```
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (455.267, 336.055)
   - 红框角度 (theta_cr): 0.471 度

>>> 正在处理: 23.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (115.164, 254.187)
   - 匹配得分 (NCC值)   : 0.927985
   - 算法耗时           : 137.423 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (111.459, 270.248)
   - 红框角度 (theta_cr): 0.571 度

>>> 正在处理: 24.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (116.164, 286.187)
   - 匹配得分 (NCC值)   : 0.925459
   - 算法耗时           : 113.866 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (112.459, 302.248)
   - 红框角度 (theta_cr): 0.571 度

>>> 正在处理: 25.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (117, 318)
   - 匹配得分 (NCC值)   : 0.923269
   - 算法耗时           : 99.0367 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (113.267, 334.055)
   - 红框角度 (theta_cr): 0.471 度

>>> 正在处理: 36.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (300, 119)
   - 匹配得分 (NCC值)   : 0.99001
   - 算法耗时           : 145.145 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (296.267, 135.055)
   - 红框角度 (theta_cr): 0.471 度

>>> 正在处理: 37.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (276.164, 119.187)
   - 匹配得分 (NCC值)   : 0.985114
   - 算法耗时           : 143.285 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (272.403, 135.235)
   - 红框角度 (theta_cr): 0.371 度
```

```
>>> 正在处理: 38.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (253, 120)
   - 匹配得分 (NCC值)   : 0.981718
   - 算法耗时           : 142.994 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (249.267, 136.055)
   - 红框角度 (theta_cr): 0.471 度

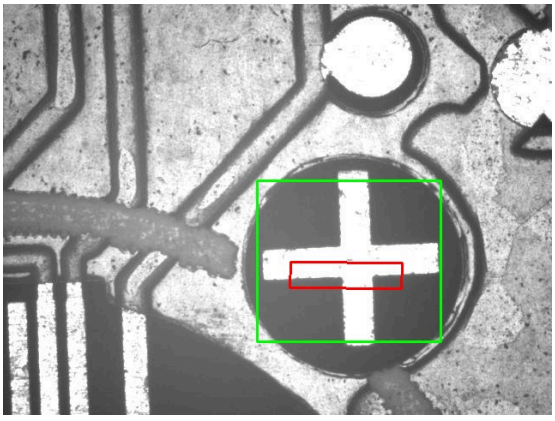
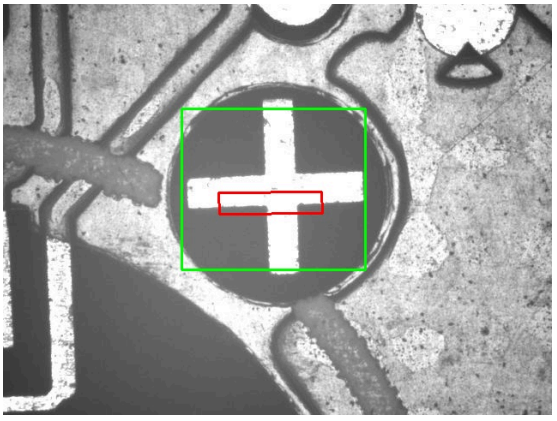
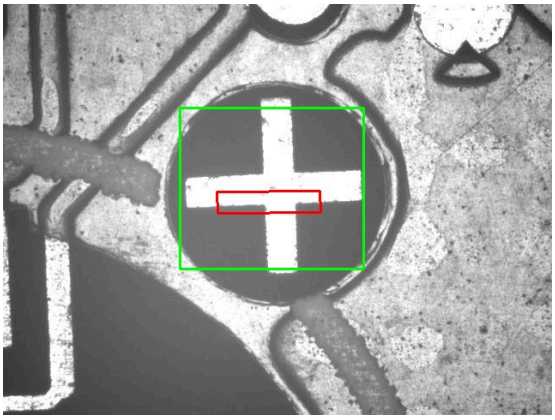
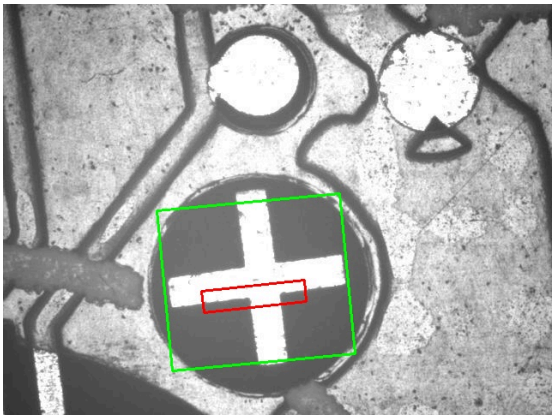
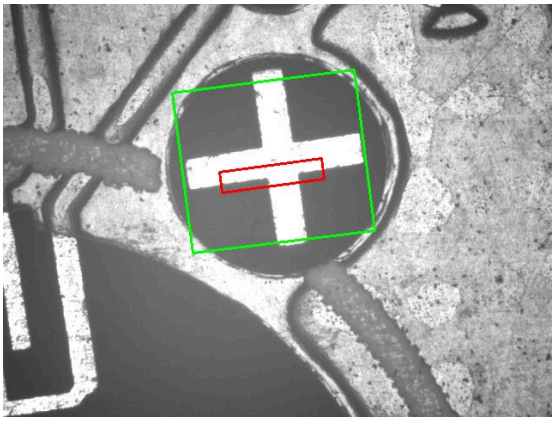
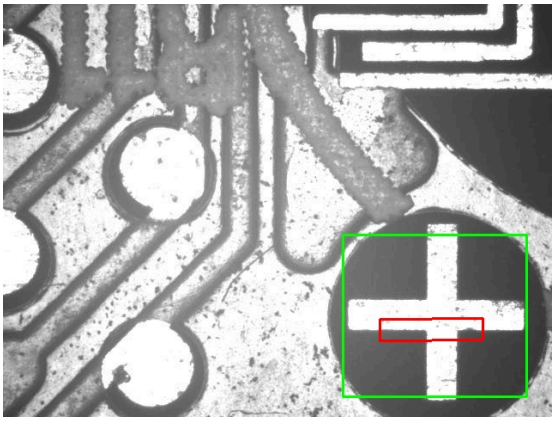
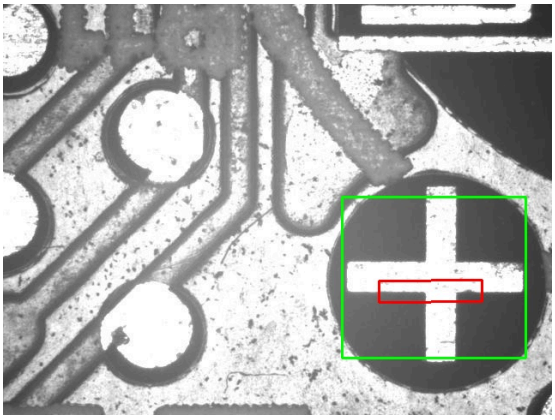
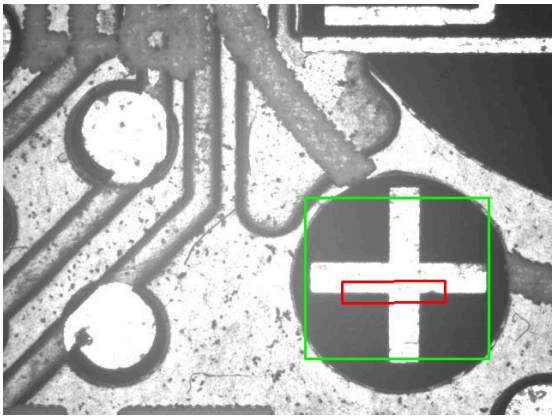
>>> 正在处理: 47.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (141.164, 338.187)
   - 匹配得分 (NCC值)   : 0.928401
   - 算法耗时           : 104.395 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (137.459, 354.248)
   - 红框角度 (theta_cr): 0.571 度

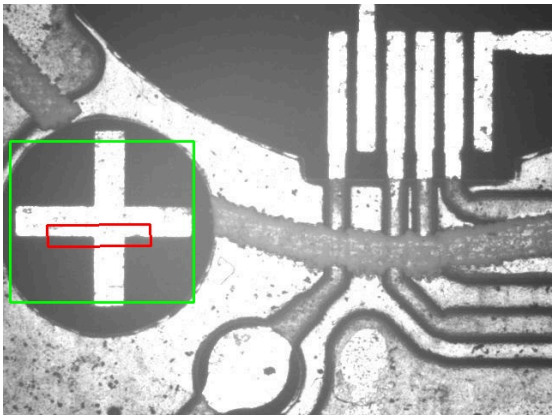
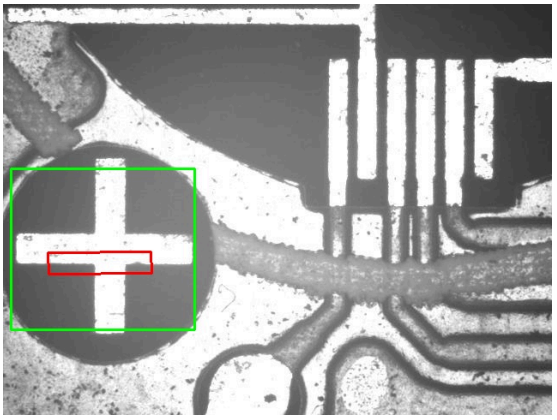
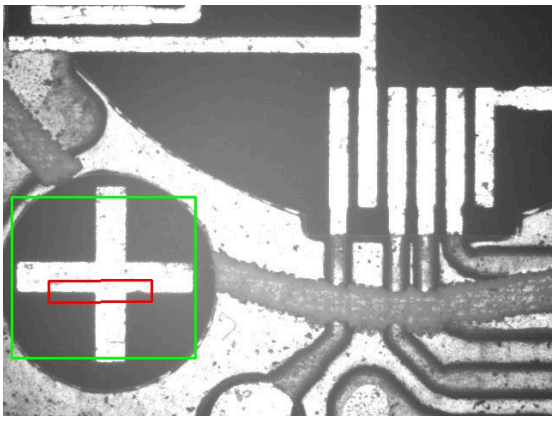
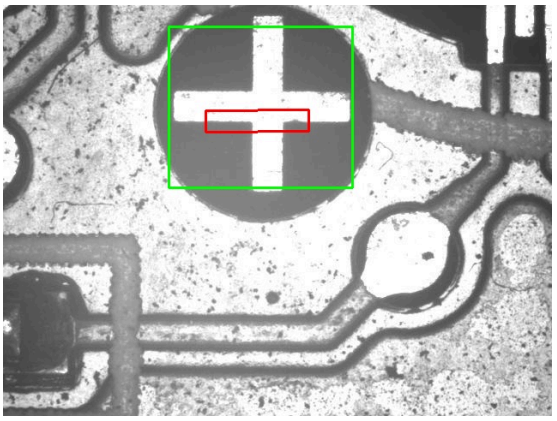
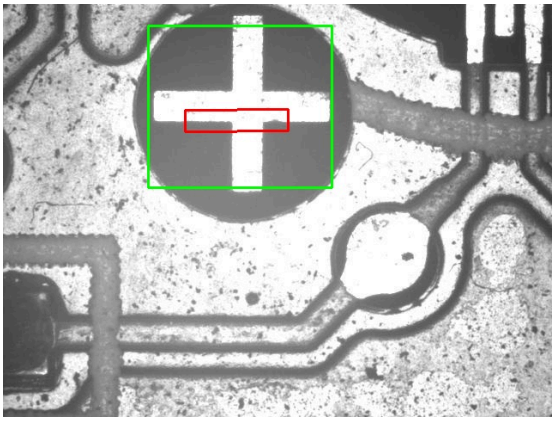
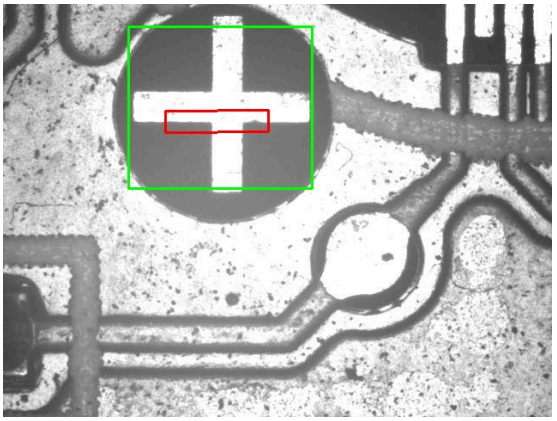
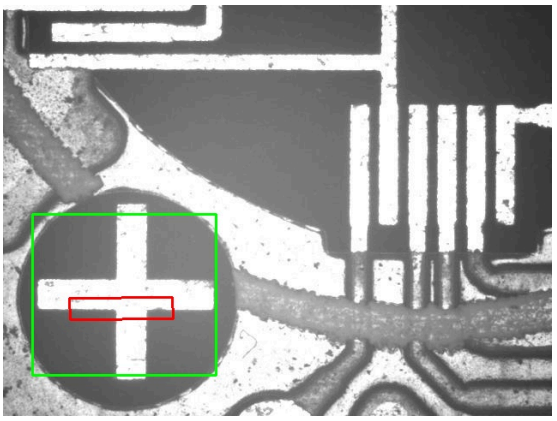
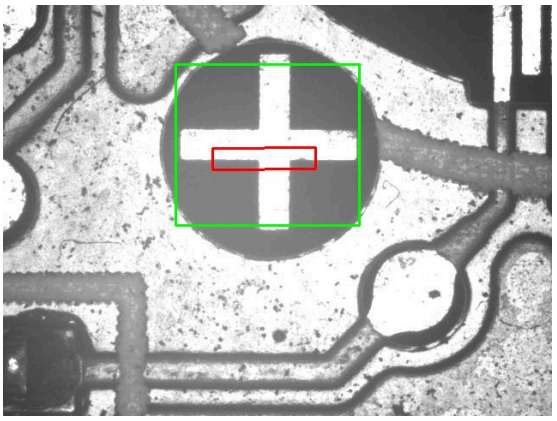
>>> 正在处理: 48.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (308, 163)
   - 匹配得分 (NCC值)   : 0.990249
   - 算法耗时           : 148.504 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (304.267, 179.055)
   - 红框角度 (theta_cr): 0.471 度

>>> 正在处理: 49.jpg <<<
  [NCC 搜索结果 (第2题)]
   - 绿框中心 (tcx, tcy): (527, 372)
   - 匹配得分 (NCC值)   : 0.983357
   - 算法耗时           : 63.1478 ms
  [ROI 推导结果 (第1题)]
   - 红框中心 (rcx, rcy): (523.267, 388.055)
   - 红框角度 (theta_cr): 0.471 度
```

五、 结果可视化展示

下方为算法自动匹配模板（绿框）并推导定位 ROI（红框）后的渲染保存图像。通过两层金字塔 NCC 及刚体仿射变换，双框在不同平移、旋转干扰下均实现了极其精准的锚定。

图像标识	结果图像	图像标识	结果图像
1-2.jpg		1-3.jpg	
1-4.jpg		1-15.jpg	
1-17.jpg		10.jpg	
11.jpg		12.jpg	

图像标识	结果图像	图像标识	结果图像
23.jpg		24.jpg	
25.jpg		36.jpg	
37.jpg		38.jpg	
47.jpg		48.jpg	

图像标识	结果图像	图像标识	结果图像
49.jpg	